untangle::executor

Generated by Doxygen 1.18.0

1 executor 1

2 Namespace Index 7
2.1 Namespace List . . . 7

3 Class Index 9
3.1 Class List . . . 9

4 File Index 11
4.1 File List . . . 11

5 Namespace Documentation 13
5.1 untangle Namespace Reference . . . 13
5.1.1 Detailed Description . . . 13
5.1.2 Enumeration Type Documentation . . . 14
5.1.2.1 task_type . . . 14
5.1.3 Variable Documentation . . . 14
5.1.3.1 unnamed_shared_pools . . . 14

6 Class Documentation 15
6.1 untangle::owned_executor< actionT > Class Template Reference . . . 15
6.1.1 Detailed Description . . . 16
6.1.2 Constructor & Destructor Documentation . . . 16
6.1.2.1 owned_executor() . . . 16
6.1.2.2 ~owned_executor() . . . 17
6.1.3 Member Function Documentation . . . 17
6.1.3.1 wait() . . . 17
6.1.3.2 add_task() . . . 18
6.1.3.3 pending() . . . 18
6.1.3.4 start() . . . 19
6.1.3.5 stop() . . . 19
6.1.4 Member Data Documentation . . . 19
6.1.4.1 on_task_error . . . 19
6.2 untangle::shared_executor< actionT > Class Template Reference . . . 20
6.2.1 Detailed Description . . . 20
6.2.2 Constructor & Destructor Documentation . . . 21
6.2.2.1 shared_executor() [1/2] . . . 21
6.2.2.2 shared_executor() [2/2] . . . 21
6.2.2.3 ~shared_executor() . . . 22
6.2.3 Member Function Documentation . . . 22
6.2.3.1 wait() . . . 22
6.2.3.2 add_task() [1/4] . . . 23
6.2.3.3 add_task() [2/4] . . . 23
6.2.3.4 add_task() [3/4] . . . 24
6.2.3.5 add_task() [4/4] . . . 25

6.2.3.6 pending() 25
6.2.3.7 start() 25
6.2.3.8 stop() 26
6.2.4 Member Data Documentation 26
6.2.4.1 on_task_error 26

7 File Documentation 27
7.1 owned_executor.hpp 27
7.2 shared_executor.hpp 31

Index 37

Chapter 1

executor

c++ thread pool executor based on async execution

A header-only library of two thread pools, each worker an async execution in continuous mode:

- owned_executor.hpp, untangle::owned_executor: a pool its owner holds alone, such as a store that runs its work on a thread of its own.
- shared_executor.hpp, untangle::shared_executor: a pool many owners share, with light and heavy workers, so a few long tasks cannot hold every worker. See the shared executor below.

Both take work the same way, and start, stop, wait and report failures the same way. The sections below hold for both, and show owned_executor.

requirements

C++23, and the async submodule:
git submodule update --init --recursive

async carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with std::println, so they need a standard library that provides <print> — GCC 14 or newer.

example

```
#include <owned_executor.hpp>
#include <functional>

int main() {
  untangle::owned_executor<std::function<void(void)» pool(4);
  pool.start();

  for (int i = 0; i < 100; ++i) {
    pool.add_task([i] { work(i); }, [] { /* finished */ });
  }

  return 0;  // ~owned_executor finishes what was added, then stops the workers
}
```

the action type

The pool is a template on the callable it runs, the way an execution is — and on the same parameter name, because every task queued is that one callable. One pool runs one signature, and the caller names it:

```
untangle::owned_executor<std::function<int(void)» pool(4);   // each result goes to the task's callback
```

The action type does not have to be a std::function. What an execution requires of it is a nested result↩ _type, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches std::function::result_type, which the standard removed in C++20 and both libc++ and libstdc++ still supply by grace rather than contract.

```
struct my_task {
  using result_type = void;
  void operator()() const;
};

untangle::owned_executor<my_task> pool(4);
```

tasks: work that reports back

Work enters the pool through one door, add_task(), and every task carries the callback it must notify. An action type may take arguments, and add_task() binds them the way its namesake execution::add_task() does — at the door, where the caller still holds them — with the callback after them:

```
untangle::owned_executor<std::function<int(int)» pool(4);
pool.start();

// the callback comes last, after the arguments the task is bound to
pool.add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

What waits for a worker is one callable carrying its own arguments and its callback, so nothing about a task changes between the queue and the worker that runs it. The arguments are copied at add_task() and handed to the task as the pool's own lvalues when it runs: a task is called later, possibly much later, and what the caller passed may be gone by then. A task taking int& therefore mutates the pool's copy, which the caller never sees.

A task that cannot be called with the arguments given is refused by a static_assert in add_task(), which names the mismatch before the errors from inside the binding follow it.

Work that returns void still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
untangle::owned_executor<std::function<void(void)» pool(4);
pool.add_task([] { /* ... */ }, [] { /* finished */ });
```

A task with nothing to bind passes the callback alone, whatever it returns:

```
untangle::owned_executor<std::function<int(void)» pool(4);
pool.add_task([] { return 42; }, [](int result) { /* result == 42 */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and untangle::bind_task() splits it off. It must be callable with the task's result and return nothing — or callable with nothing at all, when the task returns void. A missing or unusable one is a compile error rather than silence.

Work nobody wants told about still passes a callback, one that ignores what it is given. There is no fire-and-forget door: a void task's callback costs one empty call, and a single door means a single queue, in which the order out is the order in.

```
constexpr auto ignore_result = [](auto&&...) {};   // takes a result, or nothing for a void task
pool.add_task([](int n) { return n * 2; }, 21, ignore_result);
```

A refused task does not notify. add_task() answers false before start(), after stop(), while a wait() is draining, once the destructor has begun, or if the worker it was offered to has stopped — and a refused

task is destroyed rather than queued. The answer is the whole report, so a caller waiting on the callback rather than on the answer would wait for ever. The same is true of work still queued when a stopped pool is destroyed: it never runs, so it never notifies, and the count on stderr is all there is.

Finished does not mean failed. A task that throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to on_task_error. And because a callback runs inside the same try as the task that owns it, on_task_error can fire for a task that in fact succeeded — the work was done and only the telling failed.

A callback runs on a worker's thread, inside that worker's drain, and several may be running at once on different workers. It is the same rule on_task_error carries, for the same reason: nothing may escape it.

how work is placed

N executions run in continuous mode behind a shared std::deque of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — on_finished, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. execution::is_busy() is what add_task() reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

the shared executor

shared_executor is the pool to share: the application builds and starts it, and hands it to every owner by reference. It has everything above, and workers of two kinds.

light and heavy workers

A shared pool has light workers, and may have heavy ones too. untangle::task_type says which kind may run a task:

```
#include <shared_executor.hpp>

untangle::shared_executor<std::function<void(void)» pool(2, 2, "app");  // 2 light, 2 heavy
pool.start();

pool.add_task(untangle::task_type::heavy, [] { count_primes(); }, [] {});
pool.add_task(untangle::task_type::light, [] { read_limit(); }, [] {});
pool.add_task([] { anything(); }, [] {});  // untyped: any worker
```

A typed task goes to a worker of its type, picked as above within that type. Long tasks then hold only the heavy workers, and light ones keep running beside them. An untyped task goes to any worker, as before.

shared_executor(worker_count, name) builds light workers only. A heavy task on such a pool, or on one built with no heavy workers, runs on a light worker, and the pool warns once on stderr that it should be built with (light_count, heavy_count). A pool with no light worker is refused, as one with no worker is.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; start() runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign on_task_error, before anything can throw.

stop() stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. stop() does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

start() after stop() restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

wait() is a barrier, not a shutdown. It returns once everything added has run and no worker is busy, and it leaves the pool exactly as it found it — workers still in their threads, nothing destroyed, accepting again. Work added after it runs as it did before, and calling it repeatedly is how you step through work in phases:

```
pool.add_task([] { first(); }, [] {});
pool.wait();                          // first() has run, the pool is still alive
pool.add_task([] { second(); }, [] {});
pool.wait();                          // and so has second()
```

It refuses new work while it waits, then restores accepting to whatever it was. A task that re-adds itself would otherwise keep the drain from ever ending, so add_task() answers false for the duration — and because the previous state is restored rather than assumed, wait() on a stopped pool does not leave it accepting.

So only work accepted before the wait began is what the wait covers. Work offered during it was never queued, and work added after it returns is simply the next batch, for the next wait(). This is a stop-the-world drain, not a transparent one: a producer on another thread sees add_task() answer false while a wait() is in progress, and has to offer the work again afterwards.

A pool that is not running is not waited for. Only a worker takes from the queue, so a stopped pool can never empty it, and a wait insisting otherwise would never return. wait() returns at once instead, leaving what is queued for the next start().

lifetime and failure

The destructor finishes what was added. Its drain is wait(), so a task added before the pool goes out of scope has run by the time it does. What the destructor adds is the shutdown wait() deliberately leaves out, in the order that makes it safe: stop the workers, wait for every one of them to leave its thread, and only then destroy them.

The destructor waits for the pool, not for what the pool's work touched. That distinction is the whole reason wait() is public. The drain guarantees no worker is still running when the pool is destroyed, which is a statement about the pool's own members — a destructor body runs before them, so they are all still alive for it. It says nothing about the lifetime of what a task captured.

For a pool that is a local variable outliving what its work touches, the destructor is enough and nothing else is needed. For a pool held as a member of another class, it is already too late: the pool's destructor runs during its owner's member destruction, so by then the owner's derived subobject is gone, and so is every member declared after the pool. A callback reaching into either of those runs against destroyed storage however faithfully the pool drains — and on the stack that is not even a fault a sanitizer can name, because the memory is still addressable.

So an owner whose callbacks reach back into itself calls wait() where the whole object graph is still standing — in its own destructor body, ahead of its members, or better still at a point the owner of its

peers controls — and the destructor's drain goes back to being a backstop rather than the thing relied upon.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on stderr and dropped.

Both waits are unbounded, and both say so while they last. A task that never returns would otherwise leave the destructor silent for ever, so it reports instead — the first wait naming who is still inside a task, the second naming whose thread has not left. The first line below is the drain's, so a plain wait() prints it exactly as a destructor does; the second is the destructor's alone:

```
executor: still waiting to drain after 1s - 1 queued, 2 in a task: pool_1_worker_0, pool_1_worker_1
executor: still waiting to drain after 3s - 0 queued, 1 in a task: pool_1_worker_1
executor: still waiting to stop after 1s - 1 not left its thread: pool_1_worker_1
```

The interval starts at a second and doubles to a ceiling of thirty, so a stuck teardown says something almost at once without a long one becoming a flood. The two waits are kept apart because they answer different questions: a worker can be idle and still be in its thread.

Bounding either wait was never available. Returning early while a worker is still in a task is a use-after-free, and a bounded wait only moves the hang into ~execution(), which spins on its own running state with no timeout. Workers are detached and async::execution has no cancellation, so a pool cannot outlive a task it cannot interrupt. What can be fixed is the silence.

add_task() says whether the task was taken. It returns false before start(), after stop(), while a wait() is draining, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool draining is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the drain from ever ending, and that holds for a wait() just as for a destructor. One rule instead: a pool that is not running takes no work, whoever is asking. add_task() answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; on_task_error is where the throw goes:

```
untangle::owned_executor<std::function<void(void)» pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
  try {
    std::rethrow_exception(thrown);
  } catch (const std::exception& e) {
    log(e.what());
  }
};
```

It carries a std::exception_ptr because a task may throw something that is not a std::exception, and that is the case most worth hearing about. Assign it before the first add_task() — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on stderr instead, naming the worker.

The name says which pool too. An owned pool's workers are named <pool>_worker_<i>, after the name the pool was given - owned_executor pool(2, "store_a") has store_a_worker_0 and store_a_worker_1 - or, without one, pool_<n>, numbered across the process. A shared pool's are <pool>_light_<i> and <pool>_heavy_<i>, and without a name it is shared_pool_<n>. Two pools never share a worker name, so every warning - the failure reports above and the destructor's waits alike - says where it came from.

A return value is not collected by the pool: a continuous worker keeps none. It goes to the task's own callback — see tasks: work that reports back above.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

test/CMakeLists.txt fetches googletest at configure time, so the first configure needs network access.

sanitizers

EXECUTOR_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

tools/make_doc.sh builds doc/refman.pdf from the two headers and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an obsolete-tag notice, so an undocumented member or a broken \ref does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle
A thread pool its owner holds alone, built out of untangle::async::execution objects . 13

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::owned_executor< actionT >
: Runs tasks on a fixed pool of untangle::async::execution workers 15
untangle::shared_executor< actionT >
: Runs tasks on a fixed pool of untangle::async::execution workers 20

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

owned_executor.hpp 27
shared_executor.hpp 31

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool its owner holds alone, built out of untangle::async::execution objects.

Classes

- class owned_executor
  Runs tasks on a fixed pool of untangle::async::execution workers.
- class shared_executor
  Runs tasks on a fixed pool of untangle::async::execution workers.

Enumerations

- enum class task_type : std::uint8_t
  Which workers may run a task: a pool has light workers, and may have heavy ones too.

Variables

- std::atomic< std::size_t > unnamed_pools = {0}
  Numbers the pools that are not given a name, across every task type, so no two are alike.
- std::atomic< std::size_t > unnamed_shared_pools = {0}

5.1.1 Detailed Description

A thread pool its owner holds alone, built out of untangle::async::execution objects.

A thread pool many owners share, built out of untangle::async::execution objects.

N workers in continuous mode behind a shared queue. Nothing overtakes what is already queued.

5.1.2 Enumeration Type Documentation

5.1.2.1 task_type

enum class untangle::task_type : std::uint8_t [strong]

Which workers may run a task: a pool has light workers, and may have heavy ones too.

Heavy work goes to heavy workers, so a few long tasks cannot hold every worker while short ones wait behind them.

5.1.3 Variable Documentation

5.1.3.1 unnamed_shared_pools

std::atomic<std::size_t> untangle::unnamed_shared_pools = {0} [inline]

Numbers the shared pools that are not given a name, so no two are alike. Not unnamed_pools, which owned_executor.hpp defines.

Chapter 6

Class Documentation

6.1 untangle::owned_executor< actionT > Class Template Reference

Runs tasks on a fixed pool of untangle::async::execution workers.

#include <owned_executor.hpp>

Public Member Functions

- owned_executor (std::size_t worker_count, std::string name={})
  Builds worker_count workers. They do not run until start().
- owned_executor (const owned_executor &)=delete
  Not copied or moved: every worker's callbacks hold this pool's this.
- ~owned_executor ()
  Drains what was added, then stops the workers and destroys them.
- void wait ()
  Drains everything added so far, then leaves the pool as it was found.
- template<typename... Args>
  bool add_task (const actionT &task, Args &&... args)
  Gives work that must report to a worker, which queues it.
- template<typename... Args>
  bool add_task (actionT &&task, Args &&... args)
  Gives work that must report to a worker as the overload above does, moving the task into the queue rather than copying it. A temporary and a lambda land here.
- std::size_t pending () const
  How many tasks are waiting in the workers' queues, not yet taken to run.
- void start ()
  Starts every worker, and starts the pool accepting work.
- void stop ()
  Stops every worker. What they are already running, they finish.

Public Attributes

- std::function< void(std::exception_ptr)> on_task_error
  Called with whatever the work threw. Assigned by the caller.

6.1.1 Detailed Description

template<typename actionT>
class untangle::owned_executor< actionT >

Runs tasks on a fixed pool of untangle::async::execution workers.

add_task() is the one door: it carries a callback and reports the result to it.

Each worker keeps its own queue. A task goes to a worker when it is submitted - the first idle one, else the one given the fewest since it last drained - and waits there for that worker.

Template Parameters

actionT	- The callable the pool runs. One pool runs one signature, arguments and result.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 owned_executor()

template<typename actionT>
untangle::owned_executor< actionT >::owned_executor (
std::size_t worker_count,
std::string name = {}) [inline], [explicit]

Builds worker_count workers. They do not run until start().

Parameters

worker_count	- How many workers to run. Must not be zero.
name	- What its workers are named after, <name>_worker_<i>, in every warning they print. Left empty, it is pool_<n>, numbered across the process, so two pools are never alike.

Exceptions

std::invalid_argument	- worker_count is zero: such a pool could never run its work.

6.1.2.2 ~owned_executor()

template<typename actionT>
untangle::owned_executor< actionT >::~owned_executor () [inline]

Drains what was added, then stops the workers and destroys them.

The drain is wait(); the shutdown after it stops the workers, waits for each to leave its thread, and destroys them only then. Both waits are unbounded and report on stderr meanwhile.

Remarks

Nothing is dropped: a worker runs what it was given before it leaves its thread, a pool stopped earlier included.

Attention

This waits for the pool, not for what its work touched: see wait().

6.1.3 Member Function Documentation

6.1.3.1 wait()

template<typename actionT>
void untangle::owned_executor< actionT >::wait () [inline]

Drains everything added so far, then leaves the pool as it was found.

Drained means no worker has anything queued or running, so every task accepted before this has run, and so has the callback of one - a callback runs inside the worker's drain, not after it.

A barrier, not a shutdown: the workers keep their threads and work added afterwards runs as before. Reusable, and one at a time - a second caller drains once the first has returned.

The wait is unbounded and reports on stderr while it lasts.

Attention

New work is refused meanwhile, then accepting is restored, so a task that re-adds itself cannot keep the drain from ending. Only work accepted before it began counts.

Remarks

A stopped pool is waited for too: its workers run what they were given before they leave, so the wait ends.

Attention

It waits for the work, not for what the work touched. Held as a member, ~owned_executor() runs once the owner's derived parts and later members are gone: call this before that.

6.1.3.2 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::owned_executor< actionT >::add_task (
                const actionT & task,
                Args &&... args)   [inline]
```

Gives work that must report to a worker, which queues it.

The first idle worker takes it, the warmest one; otherwise the worker given the fewest since it last drained, and the task waits behind that worker's queue - in the order it arrived there. Once the task has run, its callback is invoked with the result, on the worker's thread.

Attention

The last argument is the callback, which the signature cannot say: it arrives in args and untangle::↩ bind_task() splits it off. A missing or unusable one is a static_assert, and so is a task that cannot be called with the arguments before it.

A task that throws is not notified - what it threw goes to on_task_error - and a refused one does not notify either, so a caller waiting on the callback alone waits for ever.

An empty callback or action is refused here too, silently: the answer is the report. Only a run time check can see it, and a task that can neither run nor notify is turned away before any worker is asked.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.

Parameters

task	- The task to run, copied into the worker's queue.
args	- What to bind, followed by the callback. Copied by the worker's queue, so a task taking a reference gets the pool's copy.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused by a pool not started, stopped, draining or being destroyed, by a stopped worker, or because the task could never do its job. It will not run or notify.

6.1.3.3 pending()

```
template<typename actionT>
std::size_t untangle::owned_executor< actionT >::pending () const   [inline]
```

How many tasks are waiting in the workers' queues, not yet taken to run.

Remarks

Advisory: true the moment it is read.

6.1.3.4 start()

template<typename actionT>
void untangle::owned_executor< actionT >::start () [inline]

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and again after stop() until it is called anew, which revives the workers too. Harmless twice over. wait() needs nothing from here.

6.1.3.5 stop()

template<typename actionT>
void untangle::owned_executor< actionT >::stop () [inline]

Stops every worker. What they are already running, they finish.

What the workers were given they still run, then leave; anything added afterwards is refused, a running task included. start() revives the workers; harmless twice over.

Attention

It does not wait for the workers to leave their threads - only the destructor does, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 on_task_error

template<typename actionT>
std::function<void(std::exception_ptr)> untangle::owned_executor< actionT >::on_task_error

Called with whatever the work threw. Assigned by the caller.

The only report of failed work, since add_task() answered long before it ran. Left unset, the pool warns on stderr. Rethrow the exception_ptr to read it.

Remarks

A task's failure arrives here, and so does its callback's - so this can fire for a task that succeeded, when only the telling failed.

Attention

It runs on a worker's thread, several at once. Assign it before the first add_task(), and let nothing escape it or a callback.

The documentation for this class was generated from the following file:

- owned_executor.hpp

6.2 untangle::shared_executor< actionT > Class Template Reference

Runs tasks on a fixed pool of untangle::async::execution workers.

#include <shared_executor.hpp>

Public Member Functions

- shared_executor (std::size_t worker_count, std::string name={})
  Builds worker_count light workers, and no heavy one. They do not run until start().
- shared_executor (std::size_t light_count, std::size_t heavy_count, std::string name={})
  Builds light_count light workers and heavy_count heavy ones. They do not run until start().
- shared_executor (const shared_executor &)=delete
  Not copied or moved: every worker's callbacks hold this pool's this.
- ~shared_executor ()
  Drains what was added, then stops the workers and destroys them.
- void wait ()
  Drains everything added so far, then leaves the pool as it was found.
- template<typename... Args>
  bool add_task (const actionT &task, Args &&... args)
  Gives work that must report to a worker, which queues it.
- template<typename... Args>
  bool add_task (actionT &&task, Args &&... args)
  Gives work that must report to a worker as the overload above does, moving the task into the queue rather than copying it. A temporary and a lambda land here.
- template<typename... Args>
  bool add_task (task_type type, const actionT &task, Args &&... args)
  Gives work that must report to a worker of type, as the untyped door does to any worker: the first idle one of that type, else the one of that type given the fewest.
- template<typename... Args>
  bool add_task (task_type type, actionT &&task, Args &&... args)
  Gives work that must report to a worker of type as the overload above does, moving the task into the queue rather than copying it. A temporary and a lambda land here.
- std::size_t pending () const
  How many tasks are waiting in the workers' queues, not yet taken to run.
- void start ()
  Starts every worker, and starts the pool accepting work.
- void stop ()
  Stops every worker. What they are already running, they finish.

Public Attributes

- std::function< void(std::exception_ptr)> on_task_error
  Called with whatever the work threw. Assigned by the caller.

6.2.1 Detailed Description

template<typename actionT>
class untangle::shared_executor< actionT >

Runs tasks on a fixed pool of untangle::async::execution workers.

add_task() is the one door: it carries a callback and reports the result to it.

Template Parameters

actionT	- The callable the pool runs. One pool runs one signature, arguments and result.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 shared_executor() [1/2]

```
template<typename actionT>
untangle::shared_executor< actionT >::shared_executor (
        std::size_t worker_count,
        std::string name = {})  [inline], [explicit]
```

Builds worker_count light workers, and no heavy one. They do not run until start().

Parameters

worker_count	- How many workers to run. Must not be zero.
name	- What its workers are named after, <name>_light_<i>, in every warning they print. Left empty, it is shared_pool_<n>, numbered across the process, so two pools are never alike.

Exceptions

std::invalid_argument	- worker_count is zero: such a pool could never run its work.

6.2.2.2 shared_executor() [2/2]

```
template<typename actionT>
untangle::shared_executor< actionT >::shared_executor (
        std::size_t light_count,
        std::size_t heavy_count,
        std::string name = {})  [inline]
```

Builds light_count light workers and heavy_count heavy ones. They do not run until start().

Parameters

light_count	- How many light workers to run. Must not be zero: an untyped or light task could never run.
heavy_count	- How many heavy workers to run. May be zero: heavy tasks then run on the light workers, and the pool warns once.

name	- What its workers are named after, <name>_light_<i> and <name>_heavy_<i>, in every warning they print. Left empty, it is shared_pool_<n>, numbered across the process.

Exceptions

std::invalid_argument	- light_count is zero.

6.2.2.3 ~shared_executor()

template<typename actionT>
untangle::shared_executor< actionT >::~shared_executor () [inline]

Drains what was added, then stops the workers and destroys them.

The drain is wait(); the shutdown after it stops the workers, waits for each to leave its thread, and destroys them only then. Both waits are unbounded and report on stderr meanwhile.

Remarks

Nothing is dropped: a worker runs what it was given before it leaves its thread, a pool stopped earlier included.

Attention

This waits for the pool, not for what its work touched: see wait().

6.2.3 Member Function Documentation

6.2.3.1 wait()

template<typename actionT>
void untangle::shared_executor< actionT >::wait () [inline]

Drains everything added so far, then leaves the pool as it was found.

Drained means no worker has anything queued or running, so every task accepted before this has run, and so has the callback of one - a callback runs inside the worker's drain, not after it.

A barrier, not a shutdown: the workers keep their threads and work added afterwards runs as before. Reusable, and one at a time - a second caller drains once the first has returned.

The wait is unbounded and reports on stderr while it lasts.

Attention

New work is refused meanwhile, then accepting is restored, so a task that re-adds itself cannot keep the drain from ending. Only work accepted before it began counts.

Remarks

A stopped pool is waited for too: its workers run what they were given before they leave, so the wait ends.

Attention

It waits for the work, not for what the work touched. Held as a member, ~shared_executor() runs once the owner's derived parts and later members are gone: call this before that.

6.2.3.2 add_task() [1/4]

```
template<typename actionT>
template<typename... Args>
bool untangle::shared_executor< actionT >::add_task (
                const actionT & task,
                Args &&... args)  [inline]
```

Gives work that must report to a worker, which queues it.

The first idle worker takes it, the warmest one; otherwise the worker given the fewest since it last drained, and the task waits behind that worker's queue - in the order it arrived there. Once the task has run, its callback is invoked with the result, on the worker's thread.

Attention

The last argument is the callback, which the signature cannot say: it arrives in args and untangle::↩ bind_task() splits it off. A missing or unusable one is a static_assert, and so is a task that cannot be called with the arguments before it.

A task that throws is not notified - what it threw goes to on_task_error - and a refused one does not notify either, so a caller waiting on the callback alone waits for ever.

An empty callback or action is refused here too, silently: the answer is the report. Only a run time check can see it, and a task that can neither run nor notify is turned away before any worker is asked.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.

Parameters

task	- The task to run, copied into the worker's queue.
args	- What to bind, followed by the callback. Copied by the worker's queue, so a task taking a reference gets the pool's copy.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused by a pool not started, stopped, draining or being destroyed, by a stopped worker, or because the task could never do its job. It will not run or notify.

6.2.3.3 add_task() [2/4]

```
template<typename actionT>
template<typename... Args>
bool untangle::shared_executor< actionT >::add_task (
                actionT && task,
                Args &&... args)  [inline]
```

Gives work that must report to a worker as the overload above does, moving the task into the queue rather than copying it. A temporary and a lambda land here.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.

Parameters

task	- The task to run, moved into the worker's queue.
args	- What to bind, followed by the callback, as for the overload above.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused, for the same reasons as the overload above. It will not run or notify.

6.2.3.4 add_task() [3/4]

```
template<typename actionT>
template<typename... Args>
bool untangle::shared_executor< actionT >::add_task (
                task_type type,
                const actionT & task,
                Args &&... args)   [inline]
```

Gives work that must report to a worker of type, as the untyped door does to any worker: the first idle one of that type, else the one of that type given the fewest.

A heavy task on a pool built with no heavy workers runs on a light one, and the pool warns once, the first time, that it should be built with heavy workers.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.

Parameters

type	- Which workers may run it.
task	- The task to run, copied into the worker's queue.
args	- What to bind, followed by the callback, as for the untyped door.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused, as by the untyped door.

6.2.3.5 add_task() [4/4]

```
template<typename actionT>
template<typename... Args>
bool untangle::shared_executor< actionT >::add_task (
                task_type type,
                actionT && task,
                Args &&... args)   [inline]
```

Gives work that must report to a worker of type as the overload above does, moving the task into the queue rather than copying it. A temporary and a lambda land here.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.

Parameters

type	- Which workers may run it.
task	- The task to run, moved into the worker's queue.
args	- What to bind, followed by the callback, as for the untyped door.

Returns

true - accepted, and the callback will be notified once it has run.

false - refused, as by the untyped door.

6.2.3.6 pending()

```
template<typename actionT>
std::size_t untangle::shared_executor< actionT >::pending () const   [inline]
```

How many tasks are waiting in the workers' queues, not yet taken to run.

Remarks

Advisory: true the moment it is read.

6.2.3.7 start()

```
template<typename actionT>
void untangle::shared_executor< actionT >::start ()   [inline]
```

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and again after stop() until it is called anew, which revives the workers too. Harmless twice over. wait() needs nothing from here.

6.2.3.8 stop()

template<typename actionT>
void untangle::shared_executor< actionT >::stop () [inline]

Stops every worker. What they are already running, they finish.

What the workers were given they still run, then leave; anything added afterwards is refused, a running task included. start() revives the workers; harmless twice over.

Attention

It does not wait for the workers to leave their threads - only the destructor does, and that wait is what makes destroying the pool safe.

6.2.4 Member Data Documentation

6.2.4.1 on_task_error

template<typename actionT>
std::function<void(std::exception_ptr)> untangle::shared_executor< actionT >::on_task_error

Called with whatever the work threw. Assigned by the caller.

The only report of failed work, since add_task() answered long before it ran. Left unset, the pool warns on stderr. Rethrow the exception_ptr to read it.

Remarks

A task's failure arrives here, and so does its callback's - so this can fire for a task that succeeded, when only the telling failed.

Attention

It runs on a worker's thread, several at once. Assign it before the first add_task(), and let nothing escape it or a callback.

The documentation for this class was generated from the following file:

- shared_executor.hpp

Chapter 7

File Documentation

7.1 owned_executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00008 #ifndef UNTANGLE_EXECUTOR_OWNED_EXECUTOR_HPP
00009 #define UNTANGLE_EXECUTOR_OWNED_EXECUTOR_HPP
00010
00011 #include <algorithm>
00012 #include <async.hpp>
00013 #include <atomic>
00014 #include <chrono>
00015 #include <condition_variable>
00016 #include <cstddef>
00017 #include <exception>
00018 #include <functional>
00019 #include <memory>
00020 #include <mutex>
00021 #include <print>
00022 #include <stdexcept>
00023 #include <string>
00024 #include <thread>
00025 #include <tuple>
00026 #include <type_traits>
00027 #include <utility>
00028 #include <vector>
00029
00030 namespace untangle {
00031
00033 inline std::atomic<std::size_t> unnamed_pools = {0};
00034
00045 template <typename actionT>
00046 class owned_executor {
00047  public:
00056   explicit owned_executor(std::size_t worker_count, std::string name = {}) {
00057     if (worker_count == 0) {
00058       throw std::invalid_argument("executor: worker_count must not be zero");
00059     }
00060
00061     if (name.empty()) {
00062       name = "pool_" + std::to_string(unnamed_pools.fetch_add(1) + 1);
00063     }
00064
00065     workers_.reserve(worker_count);
00066     given_ = std::make_unique<std::atomic<std::size_t>[]>(worker_count);
00067
00068     for (std::size_t index = 0; index < worker_count; ++index) {
00069       auto next = worker_execution::create_instance(name + "_worker_" + std::to_string(index));
00070
00071       // Tells the pool this worker has run everything it was given.
00072       next->on_finished = [this, index] { worker_drained(index); };
00073
00074       // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00075       next->on_error = [this, worker = next->name](std::exception_ptr thrown) {
00076         report_task_error(worker, thrown);
00077       };
00078
00079       // Adds the worker to the poll the destructor waits on.
00080       poll_.add(*next);
00081
```

```
00082       workers_.push_back(std::move(next));
00083     }
00084   }
00085
00087   owned_executor(const owned_executor&) = delete;
00088   owned_executor& operator=(const owned_executor&) = delete;
00089   owned_executor(owned_executor&&) = delete;
00090   owned_executor& operator=(owned_executor&&) = delete;
00091
00103   ~owned_executor() {
00104     // Nothing is stopped before this, or the queue could never empty. wait() leaves the pool
00105     // accepting, so stop() follows at once: that window is the only one a shutdown would take.
00106     wait();
00107     stop();
00108
00109     // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00110     // different question from the wait above: a worker can be idle and still be in its thread.
00111     // A worker leaves within microseconds of stop(), so the first tick is short and doubles up to
00112     // the ceiling: a fixed tick made every destruction sleep it once.
00113     auto interval = std::chrono::milliseconds(report_first_ms);
00114     auto waited = std::chrono::microseconds(0);
00115     auto tick = std::chrono::microseconds(poll_first_us);
00116
00117     while (poll_.is_running()) {  // time of check
00118       std::this_thread::sleep_for(tick);
00119       waited += tick;
00120       tick = std::min(tick * 2, std::chrono::microseconds(poll_max_ms * 1000));
00121
00122       if (waited >= interval) {
00123         std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00124                      std::chrono::duration_cast<std::chrono::seconds>(waited).count(),
00125                      count_workers(&worker_execution::is_running),
00126                      name_workers(&worker_execution::is_running));
00127
00128         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00129       }
00130     }
00131
00132     // time of use: no worker thread is left to reach workers_.
00133     workers_.clear();
00134   }
00135
00157   void wait() {
00158     // One drain at a time: two threads between the exchange below and the restore at the end would
00159     // read each other's cleared flag, and the last one out could leave the pool refusing work.
00160     std::lock_guard<std::mutex> drain(wait_mutex_);
00161
00162     // Put back exactly as found below: a wait must not leave a stopped pool accepting.
00163     const bool was_running = running_.exchange(false);
00164
00165     {
00166       std::unique_lock<std::mutex> lock(mutex_);
00167
00168       // Counted under the lock, before the first check: a worker that drains after this sees it and
00169       // notifies, and one that drained before it is seen by that first check.
00170       ++waiting_;
00171
00172       // Waits for every worker to go idle.
00173       auto interval = std::chrono::milliseconds(report_first_ms);
00174       auto waited = std::chrono::milliseconds(0);
00175
00176       while (!finished_cv_.wait_for(lock, interval, [this] { return nothing_running(); })) {
00177         waited += interval;
00178
00179         std::println(
00180             stderr, "executor: still waiting to drain after {}s - {} queued, {} in a task{}",
00181             waited.count() / 1000, total_pending(), count_workers(&worker_execution::is_busy),
00182             name_workers(&worker_execution::is_busy));
00183
00184         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00185       }
00186
00187       --waiting_;
00188     }
00189
00190     running_ = was_running;
00191   }
00192
00220   template <typename... Args>
00221   bool add_task(const actionT& task, Args&&... args) {
00222     // The call the pool will make, which is not quite the one written at the call site: what
00223     // reaches the task is the copies the worker binds, so that is the call that has to compile.
00224     // Without this the mismatch surfaces deep inside bind_task()'s std::apply.
00225     static_assert(invocable_with_bound<Args...>(),
00226                   "executor: the task cannot be called with the arguments given to add_task() "
00227                   "before its callback");
00228
```

```
00229     if (!running_ || task_is_empty(task, args...)) {
00230       return false;
00231     }
00232
00233     return give_to_worker(pick_worker(), task, std::forward<Args>(args)...);
00234   }
00235
00240   template <typename... Args>
00241   bool add_task(actionT&& task, Args&&... args) {
00242     static_assert(invocable_with_bound<Args...>(),
00243                   "executor: the task cannot be called with the arguments given to add_task() "
00244                   "before its callback");
00245
00246     if (!running_ || task_is_empty(task, args...)) {
00247       return false;
00248     }
00249
00250     return give_to_worker(pick_worker(), std::move(task), std::forward<Args>(args)...);
00251   }
00252
00258   std::size_t pending() const { return total_pending(); }
00259
00266   void start() {
00267     // The workers first, so anything accepted after this is handed to one that is running.
00268     for (auto& one : workers_) {
00269       one->start();
00270     }
00271
00272     running_ = true;
00273   }
00274
00285   void stop() {
00286     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00287     // worker still runs; one arriving from here is refused.
00288     running_ = false;
00289
00290     // No lock: a worker draining after stop() runs on_finished, which may take mutex_.
00291     for (auto& one : workers_) {
00292       one->stop();
00293     }
00294   }
00295
00308   std::function<void(std::exception_ptr)> on_task_error;
00309
00310 private:
00317   void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00318     if (on_task_error) {
00319       on_task_error(thrown);
00320       return;
00321     }
00322
00323     try {
00324       std::rethrow_exception(thrown);
00325     } catch (const std::exception& e) {
00326       std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00327     } catch (...) {
00328       std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00329     }
00330   }
00331
00341   template <typename... Args>
00342   static consteval bool invocable_with_bound() {
00343     if constexpr (sizeof...(Args) == 0) {
00344       return true;
00345     } else {
00346       return invocable_with<std::tuple<std::decay_t<Args>...>(
00347           std::make_index_sequence<sizeof...(Args) - 1>{});
00348     }
00349   }
00350
00352   template <typename tupleT, std::size_t... index>
00353   static consteval bool invocable_with(std::index_sequence<index...>) {
00354     return std::is_invocable_v<actionT, std::tuple_element_t<index, tupleT>&...>;
00355   }
00356
00357   using worker_execution = untangle::async::execution<actionT>;
00358
00364   std::size_t pick_worker() const {
00365     std::size_t best = 0;
00366     std::size_t best_count = given_[0].load(std::memory_order_relaxed);
00367
00368     for (std::size_t index = 0; index < workers_.size(); ++index) {
00369       const std::size_t count = given_[index].load(std::memory_order_relaxed);
00370       if (count == 0) {
00371         return index;
00372       }
00373       if (count < best_count) {
```

```
00374         best = index;
00375         best_count = count;
00376       }
00377     }
00378
00379     return best;
00380   }
00381
00391   template <typename action_t, typename... Args>
00392   bool give_to_worker(std::size_t index, action_t&& task, Args&&... args) {
00393     // No lock: the counts only steer pick_worker(), so a stale one costs balance, not correctness.
00394     given_[index].fetch_add(1, std::memory_order_relaxed);
00395
00396     if (workers_[index]->add_task(std::forward<action_t>(task), std::forward<Args>(args)...)) {
00397       return true;
00398     }
00399
00400     // Only a stopped worker refuses: add_task() turned an empty task away before. Not counted, or
00401     // an idle worker would look busy until it is next given work and drains.
00402     forget_given(index);
00403     return false;
00404   }
00405
00411   template <typename... Args>
00412   static bool task_is_empty(const actionT& task, const Args&... args) {
00413     if constexpr (testable_for_emptiness<actionT>) {
00414       if (!task) {
00415         return true;
00416       }
00417     }
00418
00419     if constexpr (sizeof...(Args) > 0) {
00420       const auto& callback = std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00421       if constexpr (testable_for_emptiness<std::decay_t<decltype(callback)>>) {
00422         if (!callback) {
00423           return true;
00424         }
00425       }
00426     }
00427
00428     return false;
00429   }
00430
00434   void forget_given(std::size_t index) {
00435     std::size_t count = given_[index].load(std::memory_order_relaxed);
00436     while (count > 0 &&
00437            !given_[index].compare_exchange_weak(count, count - 1, std::memory_order_relaxed)) {
00438     }
00439   }
00440
00444   void worker_drained(std::size_t index) {
00445     given_[index].store(0, std::memory_order_relaxed);
00446
00447     // The lock only for a wait(), which counts itself under it before its first check.
00448     if (waiting_.load() > 0) {
00449       std::lock_guard<std::mutex> lock(mutex_);
00450       finished_cv_.notify_all();
00451     }
00452   }
00453
00455   std::size_t total_pending() const {
00456     std::size_t total = 0;
00457     for (const auto& one : workers_) {
00458       total += one->pending();
00459     }
00460     return total;
00461   }
00462
00468   bool nothing_running() const {
00469     for (const auto& one : workers_) {
00470       if (one->is_busy()) {
00471         return false;
00472       }
00473     }
00474
00475     return true;
00476   }
00477
00483   std::size_t count_workers(bool (worker_execution::*state)() const) const {
00484     std::size_t count = 0;
00485
00486     for (const auto& one : workers_) {
00487       if ((one.get()->*state)()) {
00488         ++count;
00489       }
00490     }
00491
```

```
00492     return count;
00493   }
00494
00501   std::string name_workers(bool (worker_execution::*state)() const) const {
00502     std::string names;
00503
00504     for (const auto& one : workers_) {
00505       if ((one.get()->*state)()) {
00506         names += names.empty() ? ": " : ", ";
00507         names += one->name;
00508       }
00509     }
00510
00511     return names;
00512   }
00513
00516   static constexpr int poll_first_us = 100;
00517   static constexpr int poll_max_ms = 50;
00518
00520   static constexpr int report_first_ms = 1000;
00521   static constexpr int report_max_ms = 30000;
00522
00523   mutable std::mutex mutex_;
00524   std::condition_variable finished_cv_;
00525
00527   std::mutex wait_mutex_;
00528
00531   async::execution_poll poll_;
00532
00533   std::vector<std::shared_ptr<worker_execution» workers_;
00534
00536   std::unique_ptr<std::atomic<std::size_t>[]> given_;
00537
00539   std::atomic<std::size_t> waiting_ = {0};
00542   std::atomic_bool running_ = {false};
00543 };
00544
00545 }  // namespace untangle
00546
00547 #endif  // UNTANGLE_EXECUTOR_OWNED_EXECUTOR_HPP
```

7.2 shared_executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00008 #ifndef UNTANGLE_EXECUTOR_SHARED_EXECUTOR_HPP
00009 #define UNTANGLE_EXECUTOR_SHARED_EXECUTOR_HPP
00010
00011 #include <algorithm>
00012 #include <async.hpp>
00013 #include <atomic>
00014 #include <chrono>
00015 #include <condition_variable>
00016 #include <cstddef>
00017 #include <cstdint>
00018 #include <exception>
00019 #include <functional>
00020 #include <memory>
00021 #include <mutex>
00022 #include <print>
00023 #include <stdexcept>
00024 #include <string>
00025 #include <thread>
00026 #include <tuple>
00027 #include <type_traits>
00028 #include <utility>
00029 #include <vector>
00030
00031 namespace untangle {
00032
00035 inline std::atomic<std::size_t> unnamed_shared_pools = {0};
00036
00043 enum class task_type : std::uint8_t { light, heavy };
00044
00058 template <typename actionT>
00059 class shared_executor {
00060  public:
00071   explicit shared_executor(std::size_t worker_count, std::string name = {})
00072       : shared_executor(worker_count, 0, std::move(name)) {}
00073
00086   shared_executor(std::size_t light_count, std::size_t heavy_count, std::string name = {})
00087       : light_count_(light_count) {
00088     if (light_count == 0) {
```

```
00089       throw std::invalid_argument("executor: light_count must not be zero");
00090     }
00091
00092     if (name.empty()) {
00093       name = "shared_pool_" + std::to_string(unnamed_shared_pools.fetch_add(1) + 1);
00094     }
00095
00096     name_ = name;
00097     const std::size_t worker_count = light_count + heavy_count;
00098     workers_.reserve(worker_count);
00099     given_ = std::make_unique<std::atomic<std::size_t>[]>(worker_count);
00100
00101     // The light workers first, then the heavy ones: each type is one contiguous range.
00102     for (std::size_t index = 0; index < worker_count; ++index) {
00103       auto next = worker_execution::create_instance(
00104           index < light_count ? name + "_light_" + std::to_string(index)
00105                               : name + "_heavy_" + std::to_string(index - light_count));
00106
00107       // Tells the pool this worker has run everything it was given.
00108       next->on_finished = [this, index] { worker_drained(index); };
00109
00110       // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00111       next->on_error = [this, worker = next->name](std::exception_ptr thrown) {
00112         report_task_error(worker, thrown);
00113       };
00114
00115       // Adds the worker to the poll the destructor waits on.
00116       poll_.add(*next);
00117
00118       workers_.push_back(std::move(next));
00119     }
00120   }
00121
00123   shared_executor(const shared_executor&) = delete;
00124   shared_executor& operator=(const shared_executor&) = delete;
00125   shared_executor(shared_executor&&) = delete;
00126   shared_executor& operator=(shared_executor&&) = delete;
00127
00139   ~shared_executor() {
00140     // Nothing is stopped before this, or the queue could never empty. wait() leaves the pool
00141     // accepting, so stop() follows at once: that window is the only one a shutdown would take.
00142     wait();
00143     stop();
00144
00145     // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00146     // different question from the wait above: a worker can be idle and still be in its thread.
00147     // A worker leaves within microseconds of stop(), so the first tick is short and doubles up to
00148     // the ceiling: a fixed tick made every destruction sleep it once.
00149     auto interval = std::chrono::milliseconds(report_first_ms);
00150     auto waited = std::chrono::microseconds(0);
00151     auto tick = std::chrono::microseconds(poll_first_us);
00152
00153     while (poll_.is_running()) {  // time of check
00154       std::this_thread::sleep_for(tick);
00155       waited += tick;
00156       tick = std::min(tick * 2, std::chrono::microseconds(poll_max_ms * 1000));
00157
00158       if (waited >= interval) {
00159         std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00160                      std::chrono::duration_cast<std::chrono::seconds>(waited).count(),
00161                      count_workers(&worker_execution::is_running),
00162                      name_workers(&worker_execution::is_running));
00163
00164         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00165       }
00166     }
00167
00168     // time of use: no worker thread is left to reach workers_.
00169     workers_.clear();
00170   }
00171
00193   void wait() {
00194     // One drain at a time: two threads between the exchange below and the restore at the end would
00195     // read each other's cleared flag, and the last one out could leave the pool refusing work.
00196     std::lock_guard<std::mutex> drain(wait_mutex_);
00197
00198     // Put back exactly as found below: a wait must not leave a stopped pool accepting.
00199     const bool was_running = running_.exchange(false);
00200
00201     {
00202       std::unique_lock<std::mutex> lock(mutex_);
00203
00204       // Counted under the lock, before the first check: a worker that drains after this sees it and
00205       // notifies, and one that drained before it is seen by that first check.
00206       ++waiting_;
00207
00208       // Waits for every worker to go idle.
```

```
00209       auto interval = std::chrono::milliseconds(report_first_ms);
00210       auto waited = std::chrono::milliseconds(0);
00211
00212       while (!finished_cv_.wait_for(lock, interval, [this] { return nothing_running(); })) {
00213         waited += interval;
00214
00215         std::println(
00216             stderr, "executor: still waiting to drain after {}s - {} queued, {} in a task{}",
00217             waited.count() / 1000, total_pending(), count_workers(&worker_execution::is_busy),
00218             name_workers(&worker_execution::is_busy));
00219
00220         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00221       }
00222
00223       --waiting_;
00224     }
00225
00226     running_ = was_running;
00227   }
00228
00256   template <typename... Args>
00257   bool add_task(const actionT& task, Args&&... args) {
00258     // The call the pool will make, which is not quite the one written at the call site: what
00259     // reaches the task is the copies the worker binds, so that is the call that has to compile.
00260     // Without this the mismatch surfaces deep inside bind_task()'s std::apply.
00261     static_assert(invocable_with_bound<Args...>(),
00262                   "executor: the task cannot be called with the arguments given to add_task() "
00263                   "before its callback");
00264
00265     if (!running_ || task_is_empty(task, args...)) {
00266       return false;
00267     }
00268
00269     return give_to_worker(pick_worker(), task, std::forward<Args>(args)...);
00270   }
00271
00282   template <typename... Args>
00283   bool add_task(actionT&& task, Args&&... args) {
00284     static_assert(invocable_with_bound<Args...>(),
00285                   "executor: the task cannot be called with the arguments given to add_task() "
00286                   "before its callback");
00287
00288     if (!running_ || task_is_empty(task, args...)) {
00289       return false;
00290     }
00291
00292     return give_to_worker(pick_worker(), std::move(task), std::forward<Args>(args)...);
00293   }
00294
00309   template <typename... Args>
00310   bool add_task(task_type type, const actionT& task, Args&&... args) {
00311     static_assert(invocable_with_bound<Args...>(),
00312                   "executor: the task cannot be called with the arguments given to add_task() "
00313                   "before its callback");
00314
00315     if (!running_ || task_is_empty(task, args...)) {
00316       return false;
00317     }
00318
00319     return give_to_worker(pick_worker(type), task, std::forward<Args>(args)...);
00320   }
00321
00333   template <typename... Args>
00334   bool add_task(task_type type, actionT&& task, Args&&... args) {
00335     static_assert(invocable_with_bound<Args...>(),
00336                   "executor: the task cannot be called with the arguments given to add_task() "
00337                   "before its callback");
00338
00339     if (!running_ || task_is_empty(task, args...)) {
00340       return false;
00341     }
00342
00343     return give_to_worker(pick_worker(type), std::move(task), std::forward<Args>(args)...);
00344   }
00345
00351   std::size_t pending() const { return total_pending(); }
00352
00359   void start() {
00360     // The workers first, so anything accepted after this is handed to one that is running.
00361     for (auto& one : workers_) {
00362       one->start();
00363     }
00364
00365     running_ = true;
00366   }
00367
00378   void stop() {
```

```
00379     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00380     // worker still runs; one arriving from here is refused.
00381     running_ = false;
00382
00383     // No lock: a worker draining after stop() runs on_finished, which may take mutex_.
00384     for (auto& one : workers_) {
00385       one->stop();
00386     }
00387   }
00388
00401   std::function<void(std::exception_ptr)> on_task_error;
00402
00403  private:
00410   void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00411     if (on_task_error) {
00412       on_task_error(thrown);
00413       return;
00414     }
00415
00416     try {
00417       std::rethrow_exception(thrown);
00418     } catch (const std::exception& e) {
00419       std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00420     } catch (...) {
00421       std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00422     }
00423   }
00424
00434   template <typename... Args>
00435   static consteval bool invocable_with_bound() {
00436     if constexpr (sizeof...(Args) == 0) {
00437       return true;
00438     } else {
00439       return invocable_with<std::tuple<std::decay_t<Args>...>(
00440           std::make_index_sequence<sizeof...(Args) - 1>{});
00441     }
00442   }
00443
00445   template <typename tupleT, std::size_t... index>
00446   static consteval bool invocable_with(std::index_sequence<index...>) {
00447     return std::is_invocable_v<actionT, std::tuple_element_t<index, tupleT>&...>;
00448   }
00449
00450   using worker_execution = untangle::async::execution<actionT>;
00451
00458   std::size_t pick_worker() const { return pick_worker(0, workers_.size()); }
00459
00471   std::size_t pick_worker(task_type type) {
00472     if (type == task_type::light) {
00473       return pick_worker(0, light_count_);
00474     }
00475
00476     if (light_count_ < workers_.size()) {
00477       return pick_worker(light_count_, workers_.size());
00478     }
00479
00480     if (!heavy_fallback_warned_.exchange(true, std::memory_order_relaxed)) {
00481       std::println(stderr,
00482                    "warning: executor {}: a heavy task found no heavy workers and ran on a light "
00483                    "one - build the pool with (light_count, heavy_count) to dedicate heavy workers",
00484                    name_);
00485     }
00486     return pick_worker(0, light_count_);
00487   }
00488
00499   std::size_t pick_worker(std::size_t first, std::size_t last) const {
00500     std::size_t best = first;
00501     std::size_t best_count = given_[first].load(std::memory_order_relaxed);
00502
00503     for (std::size_t index = first; index < last; ++index) {
00504       const std::size_t count = given_[index].load(std::memory_order_relaxed);
00505       if (count == 0) {
00506         return index;
00507       }
00508       if (count < best_count) {
00509         best = index;
00510         best_count = count;
00511       }
00512     }
00513
00514     return best;
00515   }
00516
00526   template <typename action_t, typename... Args>
00527   bool give_to_worker(std::size_t index, action_t&& task, Args&&... args) {
00528     // No lock: the counts only steer pick_worker(), so a stale one costs balance, not correctness.
00529     given_[index].fetch_add(1, std::memory_order_relaxed);
```

```
00530
00531     if (workers_[index]->add_task(std::forward<action_t>(task), std::forward<Args>(args)...)) {
00532       return true;
00533     }
00534
00535     // Only a stopped worker refuses: add_task() turned an empty task away before. Not counted, or
00536     // an idle worker would look busy until it is next given work and drains.
00537     forget_given(index);
00538     return false;
00539   }
00540
00546   template <typename... Args>
00547   static bool task_is_empty(const actionT& task, const Args&... args) {
00548     if constexpr (testable_for_emptiness<actionT>) {
00549       if (!task) {
00550         return true;
00551       }
00552     }
00553
00554     if constexpr (sizeof...(Args) > 0) {
00555       const auto& callback = std::get<sizeof...(Args) - 1>(std::forward_as_tuple(args...));
00556       if constexpr (testable_for_emptiness<std::decay_t<decltype(callback)»>) {
00557         if (!callback) {
00558           return true;
00559         }
00560       }
00561     }
00562
00563     return false;
00564   }
00565
00569   void forget_given(std::size_t index) {
00570     std::size_t count = given_[index].load(std::memory_order_relaxed);
00571     while (count > 0 &&
00572            !given_[index].compare_exchange_weak(count, count - 1, std::memory_order_relaxed)) {
00573     }
00574   }
00575
00579   void worker_drained(std::size_t index) {
00580     given_[index].store(0, std::memory_order_relaxed);
00581
00582     // The lock only for a wait(), which counts itself under it before its first check.
00583     if (waiting_.load() > 0) {
00584       std::lock_guard<std::mutex> lock(mutex_);
00585       finished_cv_.notify_all();
00586     }
00587   }
00588
00590   std::size_t total_pending() const {
00591     std::size_t total = 0;
00592     for (const auto& one : workers_) {
00593       total += one->pending();
00594     }
00595     return total;
00596   }
00597
00603   bool nothing_running() const {
00604     for (const auto& one : workers_) {
00605       if (one->is_busy()) {
00606         return false;
00607       }
00608     }
00609
00610     return true;
00611   }
00612
00618   std::size_t count_workers(bool (worker_execution::*state)() const) const {
00619     std::size_t count = 0;
00620
00621     for (const auto& one : workers_) {
00622       if ((one.get()->*state)()) {
00623         ++count;
00624       }
00625     }
00626
00627     return count;
00628   }
00629
00636   std::string name_workers(bool (worker_execution::*state)() const) const {
00637     std::string names;
00638
00639     for (const auto& one : workers_) {
00640       if ((one.get()->*state)()) {
00641         names += names.empty() ? ": " : ", ";
00642         names += one->name;
00643       }
00644     }
```

```
00645
00646     return names;
00647   }
00648
00651   static constexpr int poll_first_us = 100;
00652   static constexpr int poll_max_ms = 50;
00653
00655   static constexpr int report_first_ms = 1000;
00656   static constexpr int report_max_ms = 30000;
00657
00658   mutable std::mutex mutex_;
00659   std::condition_variable finished_cv_;
00660
00662   std::mutex wait_mutex_;
00663
00666   async::execution_poll poll_;
00667
00668   std::vector<std::shared_ptr<worker_execution» workers_;
00669
00671   std::unique_ptr<std::atomic<std::size_t>[]> given_;
00672
00674   std::atomic<std::size_t> waiting_ = {0};
00677   std::atomic_bool running_ = {false};
00678
00680   std::size_t light_count_;
00681
00683   std::string name_;
00684
00686   std::atomic_bool heavy_fallback_warned_ = {false};
00687 };
00688
00689 }  // namespace untangle
00690
00691 #endif  // UNTANGLE_EXECUTOR_SHARED_EXECUTOR_HPP
```

Index

~owned_executor
  untangle::owned_executor< actionT >, 16
~shared_executor
  untangle::shared_executor< actionT >, 22

add_task
  untangle::owned_executor< actionT >, 17
  untangle::shared_executor< actionT >, 22, 23, 24

executor, 1

on_task_error
  untangle::owned_executor< actionT >, 19
  untangle::shared_executor< actionT >, 26
owned_executor
  untangle::owned_executor< actionT >, 16

pending
  untangle::owned_executor< actionT >, 18
  untangle::shared_executor< actionT >, 25

shared_executor
  untangle::shared_executor< actionT >, 21
start
  untangle::owned_executor< actionT >, 18
  untangle::shared_executor< actionT >, 25
stop
  untangle::owned_executor< actionT >, 19
  untangle::shared_executor< actionT >, 25

task_type
  untangle, 14

unnamed_shared_pools
  untangle, 14
untangle, 13
  task_type, 14
  unnamed_shared_pools, 14
untangle::owned_executor< actionT >, 15
  ~owned_executor, 16
  add_task, 17
  on_task_error, 19
  owned_executor, 16
  pending, 18
  start, 18
  stop, 19
  wait, 17
untangle::shared_executor< actionT >, 20
  ~shared_executor, 22
  add_task, 22, 23, 24
  on_task_error, 26
  pending, 25
  shared_executor, 21
  start, 25
  stop, 25
  wait, 22

wait
  untangle::owned_executor< actionT >, 17
  untangle::shared_executor< actionT >, 22